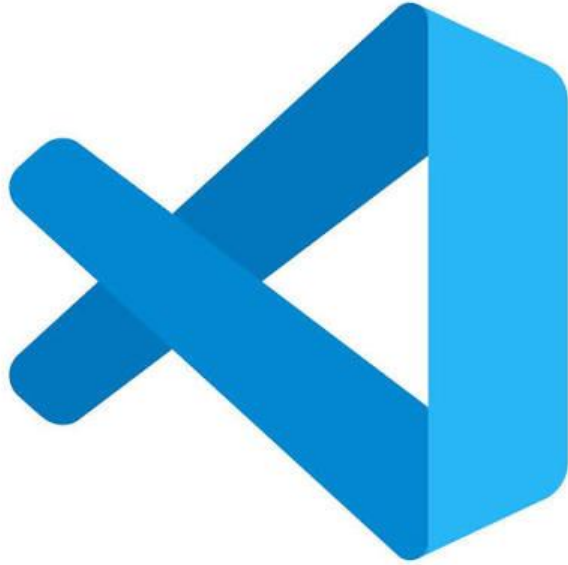


GUI 프로그래밍

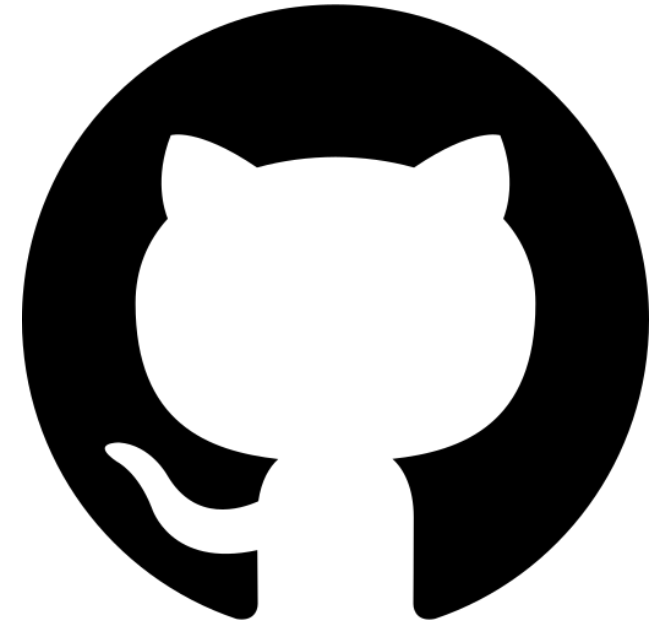
Bishal (비살)

2026-09-07

1. 환경 설정
2. 첫 CLI 프로그램
3. 첫 GUI 프로그램
4. 기본적인 창 만들기
5. 창 기본 설정
6. 위젯의 개념과 계층 구조
7. 라벨, 버튼, 입력창
8. 실습 1
9. 실습 1



[Visual Studio Code](#)

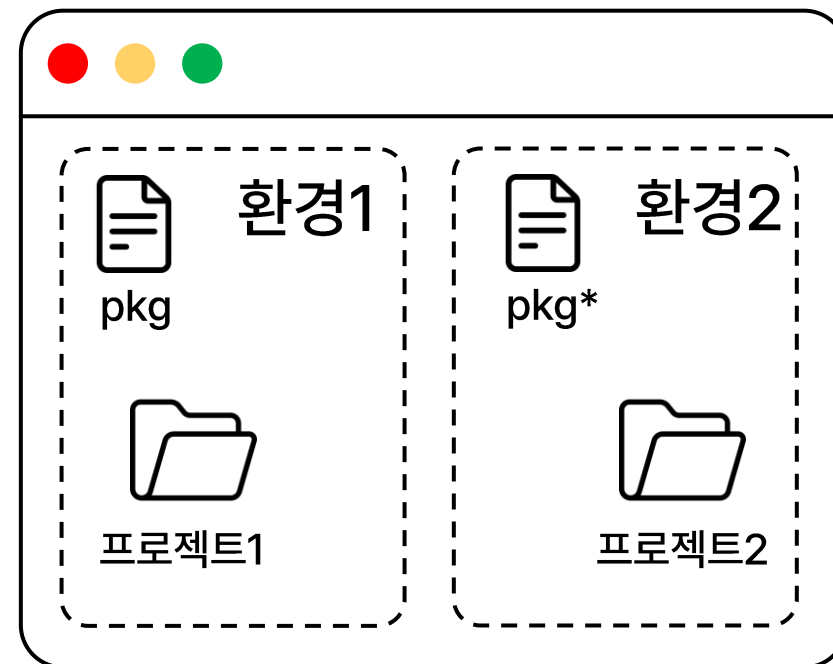


[GitHub](#)

[bluesaiyancodes/GUI-Programming: 김오공과대학교 \[GA3018-02\]](#)

환경 설정은 필요한 건가요?

- 프로젝트 격리 (Isolation): 프로젝트마다 필요한 라이브러리 버전 충돌 방지
- OS 보호 & 권한 문제 예방: 시스템 전역 Python을 오염시키지 않고 깨끗한 상태 유지
- 협업과 재현성 (Reproducibility): requirements.txt 하나로 누구나 동일한 실행 환경 구축



환경 설정하는 방법

```
python -m venv <환경>
```

```
# Windows에서
```

```
<환경>\Scripts\Activate
```

```
# Mac, Linux에서
```

```
source <환경>\Scripts\Activate
```

venv – **virtual environment** (가상환경)

alert_cli.py

```
1  import argparse
2  import sys
3  import time
4
5
6  if hasattr(sys.stdout, "reconfigure"):
7      sys.stdout.reconfigure(encoding="utf-8")
8      sys.stderr.reconfigure(encoding="utf-8")
9
10
11 def main():
12     parser = argparse.ArgumentParser(
13         description="지정한 시간이 지난 후 알림 메시지를 출력합니다."
14     )
15     parser.add_argument(
16         "seconds",
17         help="대기할 시간(초)",
18     )
19     parser.add_argument(
20         "message",
21         nargs="*",
22         help='선택적 알림 메시지(기본값: "알림!")',
23     )
24     args = parser.parse_args()
25
26     message = " ".join(args.message) if args.message else "알림!"
27
28     try:
29         secs = float(args.seconds)
30         if secs < 0:
31             raise ValueError("초는 0 이상이어야 합니다")
32         time.sleep(secs)
33     except KeyboardInterrupt:
34         print("\n알림이 취소되었습니다.")
35         return
36
37     print(message)
38
39
40 if __name__ == "__main__":
41     main()
```

```
workspace> python .\alert_cli.py 3 Time is Up
Time is Up
```

```

1  import argparse
2  import sys
3  import time
4
5
6  if hasattr(sys.stdout, "reconfigure"):
7      sys.stdout.reconfigure(encoding="utf-8")
8      sys.stderr.reconfigure(encoding="utf-8")
9
10
11 def main():
12     parser = argparse.ArgumentParser(
13         description="지정한 시간이 지난 후 알림 메시지를 출력합니다."
14     )
15     parser.add_argument(
16         "seconds",
17         help="대기할 시간(초)",
18     )
19     parser.add_argument(
20         "message",
21         nargs="*",
22         help='선택적 알림 메시지(기본값: "알림!")',
23     )
24     args = parser.parse_args()
25
26     message = " ".join(args.message) if args.message else "알림!"
27
28     try:
29         secs = float(args.seconds)
30         if secs < 0:
31             raise ValueError("초는 0 이상이어야 합니다")
32         time.sleep(secs)
33     except KeyboardInterrupt:
34         print("\n알림이 취소되었습니다.")
35         return
36
37     print(message)
38
39
40 if __name__ == "__main__":
41     main()

```

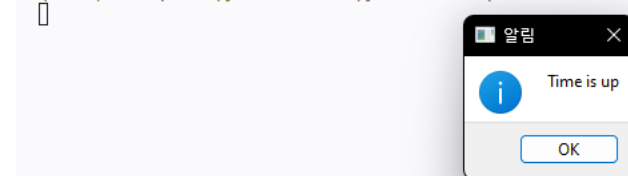
alert.py

```

1  import argparse
2  import sys
3
4  from PyQt5.QtCore import QTimer
5  from PyQt5.QtWidgets import QApplication, QMessageBox
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30  try:
31      secs = float(args.seconds)
32      if secs < 0:
33          raise ValueError
34  except ValueError:
35      parser.error("초는 0 이상의 숫자여야 합니다")
36
37  app = QApplication([sys.argv])
38
39  def show_alert():
40      QMessageBox.information(None, "알림", message)
41      app.quit()
42
43  QTimer.singleShot(int(secs * 1000), show_alert)
44  sys.exit(app.exec_())

```

○ (.venv) workspace> python .\alert.py 3 Time is up



alert.py

```

37  app = QApplication([sys.argv])
38
39  def show_alert():
40      QMessageBox.information(None, "알림", message)
41      app.quit()
42
43      QTimer.singleShot(int(secs * 1000), show_alert)
44      sys.exit(app.exec_())
    
```

sys.argv 가 뭐예요?

- QApplication은 GUI 프로그램 전체를 총괄하는 단 하나의 관리자
- 이벤트 루프 운영, 화면·폰트·색상 등 전역 설정 관리, 마우스·키보드 입력의 전달 같은 모든 밑바탕 작업을 담당함.

QApplication은 어떤 위젯보다 먼저 생성해야 합니다. 위젯은 내부적으로 이 객체에 의존하기 때문입니다.
한 프로그램에 QApplication은 하나만 존재해야 합니다.

sys.argv는 프로그램을 실행할 때 함께 넘어온 명령행 인자 목록입니다.

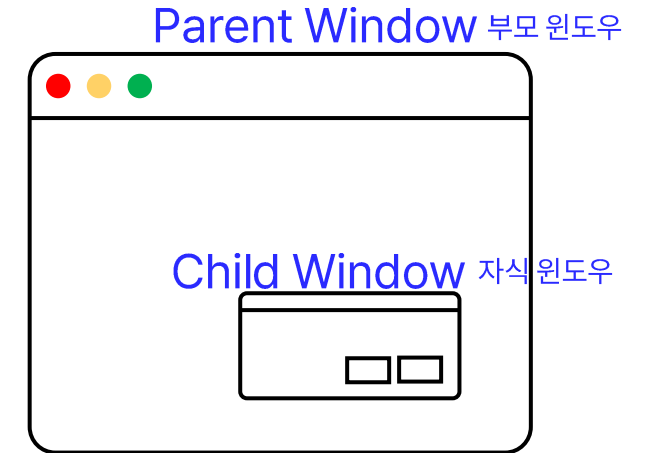
QApplication에 이를 전달하면 Qt가 인식하는 몇몇 옵션(예 : 스타일 지정)을 명령행에서 받을 수 있습니다.
인자를 쓸 일이 없다면 빈 리스트 QApplication([])를 넘겨도 동작하지만, 관례적으로 sys.argv를 넘깁니다.

alert.py

```

37 app = QApplication([sys.argv])
38
39 def show_alert():
40     QMessageBox.information(None, "알림", message)
41     app.quit()
42
43 QTimer.singleShot(int(secs * 1000), show_alert)
44 sys.exit(app.exec_())
    
```

Parent? Header 내용
 (Arrows point from these labels to the arguments in the QMessageBox.information call)



QMessageBox는 사용자에게 공지사항을 보여주거나 질문을 던지고 오류 메시지를 전달하는 미리 만들어진 대화상자 창이다.

확인이나 취소 같은 표준 버튼을 포함하고 있어 개발자가 직접 창을 처음부터 만들지 않고도 사용자에게 빠르게 알림을 줄 수 있다.

alert.py

```

37     app = QApplication([sys.argv])
38
39     def show_alert():
40         QMessageBox.information(None, "알림", message)
41         app.quit()
42
43     QTimer.singleShot(int(secs * 1000), show_alert)
44     sys.exit(app.exec_())
    
```

QTimer.singleShot은 지정된 시간이 지난 뒤에 특정 함수를 실행한다.

지연 시간을 **밀리초** 단위로 받고 실행할 함수의 이름을 입력받는다.

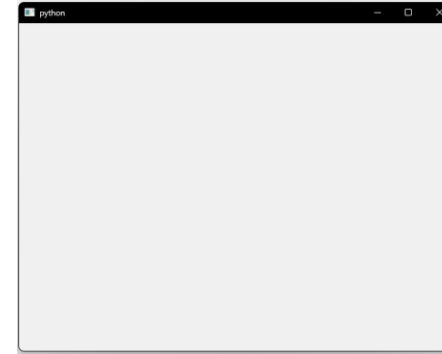
타이머가 그 시간 동안 기다린 다음 함수를 한 번 실행하고 알아서 꺼진다. 그래서 타이머 객체를 직접 멈추거나 지울 필요가 없습니다.

빈 창 하나를 띄우는 가장 기본적인 코드

qWidget.py

```
1 import sys
2 from PyQt5.QtWidgets import QApplication, QWidget
3
4 app = QApplication(sys.argv)
5 window = QWidget()
6 window.show()
7 sys.exit(app.exec())
```

창을 화면에 표시

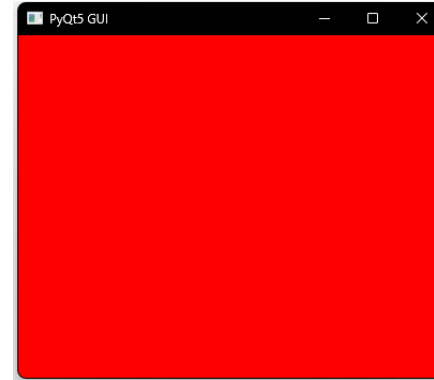


QWidget은 버튼, 입력창, 라벨, 창 같은 GUI 요소들의 기본이 되는 클래스입니다.
여기서는 아무 내용이 없는 기본 창으로 사용합니다.

제목과 크기, 배경색을 지정한 창

qWidget_modif.py

```
1  import sys
2  from PyQt5.QtWidgets import QApplication, QWidget
3
4  app = QApplication(sys.argv)
5
6  window = QWidget()
7  window.setWindowTitle("PyQt5 GUI")
8  window.resize(400, 320)
9  window.setStyleSheet("background-color: red")
10
11 window.show()
12 sys.exit(app.exec())
```



setWindowTitle(text) → 제목 표시줄에 표시할 텍스트 지정

resize(width, height) → 창의 초기 크기(픽셀) 지정

setFixedSize(w, h) → 크기를 고정해 사용자가 못 바꾸게 함

move(x, y) → 창이 뜰 화면상의 위치 지정

setStyleSheet(style) → 위젯의 스타일(배경색, 폰트 등)을 CSS 형식으로 지정

show() → 창을 화면에 표시

close() → 창을 닫음

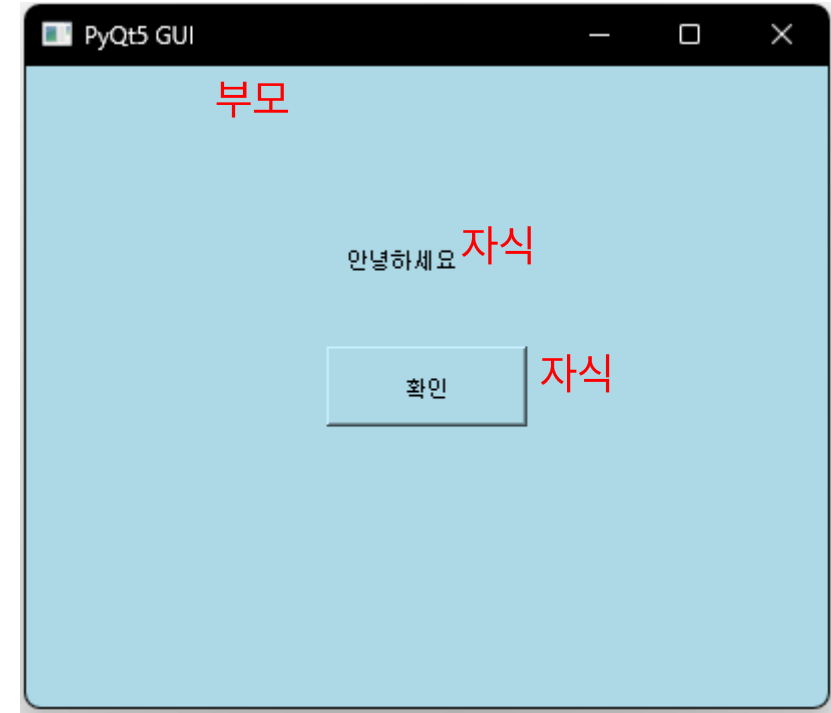
qWidget2.py

```
1  import sys
2  from PyQt5.QtWidgets import QApplication, QWidget, QPushButton, QLabel
3
4  app = QApplication(sys.argv)
5
6  window = QWidget()
7  window.setWindowTitle("PyQt5 GUI")
8  window.resize(400, 320)
9  window.setStyleSheet("background-color: lightblue")
10
11  lbl = QLabel("안녕하세요", window)
12  lbl.move(160, 90)
13
14  btn = QPushButton("확인", window)
15  btn.resize(100, 40)
16  btn.move(150, 140)
17
18  window.show()
19  sys.exit(app.exec_())
20
```



qWidget2.py

```
1  import sys
2  from PyQt5.QtWidgets import QApplication, QWidget, QPushButton, QLabel
3
4  app = QApplication(sys.argv)
5
6  window = QWidget()
7  window.setWindowTitle("PyQt5 GUI")
8  window.resize(400, 320)
9  window.setStyleSheet("background-color: lightblue")
10
11  lbl = QLabel("안녕하세요", window)
12  lbl.move(160, 90)
13
14  btn = QPushButton("확인", window)
15  btn.resize(100, 40)
16  btn.move(150, 140)
17
18  window.show()
19  sys.exit(app.exec_())
20
```



QLineEdit(window) → 입력창

대부분의 프로그램은 **무언가를 보여주고(라벨), 사용자가 누르고(버튼), 값을 입력받는(입력창)** 세 가지 동작을 기본으로 합니다. PyQt5에서는 각각 QLabel, QPushButton, QLineEdit 가 이 역할을 맡습니다.

이 장에서는 이 세 위젯의 사용법과 함께, 버튼을 눌렀을 때 동작을 연결하는 **시그널과 슬롯**의 첫걸음을 익힙니다.

라벨

글자를 보여주는 위젯

- QLabel은 화면에 텍스트나 이미지를 표시하는 가장 단순한 위젯입니다.

```
from PyQt5.QtWidgets import QApplication, QWidget, QLabel

label = QLabel("안녕하세요", window)
```

- 라벨 생성 후 setText()로 내용을 변경할 수 있습니다.
- 글자의 정렬은 setAlignment()로 지정합니다.

라벨

Method	설명	
setText(str)	라벨에 표시할 텍스트 설정	
text()	현재 텍스트 반환	
setWordWrap(bool)	자동 줄바꿈 여부 설정	
setStyleSheet(str)	CSS 스타일 적용	
setTextFormat(Qt.TextFormat)	텍스트 포맷 지정 (Plain/Rich/Markdown)	
setAlignment(Qt.Alignment)	정렬 방식 설정	Qt.AlignLeft, Qt.AlignRight, Qt.AlignCenter 등
clear()	내용 비우기	

버튼

- QPushButton은 사용자의 클릭 이벤트를 처리하는 위젯입니다. 생성 후 반드시 슬롯을 연결해야 제 역할을 합니다.

```
from PySide6.QtWidgets import QPushButton
```

```
button = QPushButton("확인", window)
```

Method	설명
setText(str)	버튼에 표시할 텍스트 설정
setStyleSheet(str)	CSS 스타일 적용
setEnabled(bool)	활성/비활성 상태 설정
setCheckable(bool)	토글(체크) 가능 여부 설정
isChecked()	체크(눌림) 상태 반환
setDefault(bool)	기본 버튼 지정 (Enter로 동작)
text()	현재 텍스트 반환

입력창

- QLineEdit은 사용자로부터 한 줄의 텍스트를 입력받는 위젯입니다.

```
from PySide6.QtWidgets import QLineEdit

le = QLineEdit(window)
le.setPlaceholderText("이름을 입력하세요")
```

Method	설명
setText(str)	텍스트 설정
text()	현재 입력된 텍스트 반환
clear()	텍스트 전체 삭제
setPlaceholderText(str)	비어 있을 때 표시할 안내 문구 설정
setEchoMode(mode)	표시 방식 설정 (Normal, Password 등)
setReadOnly(bool)	읽기 전용 여부 설정
setMaxLength(int)	최대 입력 길이 설정
setValidator(validator)	입력 검증기 설정(숫자만, 이메일 형식만 등)

목표

PyQt5를 이용해 기본적인 창을 만들고, 라벨과 버튼을 올바른 위치에 배치하는 연습을 합니다.

요구사항

1. 프로그램의 제목을 "실습 1"로 설정합니다.
2. 창의 크기는 **가로 500, 세로 400**으로 지정합니다.
3. 창의 배경색을 연한 초록색(lightgreen)으로 변경합니다.
4. 창 중앙 부근에 QLabel을 생성하여 "안녕하세요, PyQt5!"라는 텍스트를 출력합니다.
5. 라벨 아래쪽에 QPushButton을 생성하여 버튼 문구를 "클릭"으로 설정하고, 크기를 **가로 120, 세로 40**으로 지정합니다.

목표

사용자로부터 입력창(QLineEdit)을 통해 값을 받고, 위젯의 다양한 옵션과 스타일을 활용하는 연습을 합니다.

요구사항

1. 프로그램의 제목을 "실습 2 - 사용자 정보 입력"으로 설정하고, 크기는 **가로 450, 세로 350**으로 고정합니다.
(setFixedSize 활용)
2. QLineEdit 입력창을 만들고, 비어 있을 때 연한 회색 글씨로 "이름을 입력하세요"라는 안내 문구가 나오도록 설정합니다.
(setPlaceholderText 활용)
3. 라벨 하나를 만들어 초기 텍스트를 "환영합니다!"로 설정하고, 글자 정렬을 가운데 정렬(AlignCenter)로 지정합니다.
4. 버튼을 두 개 만듭니다.
 1. 첫 번째 버튼 문구: **"확인"** → 라벨 "<이름>님 환영합니다!"로 바꾸기
 2. 두 번째 버튼 문구: **"지우기"** → QLineEdit 비어 있는 상태로 들어가기, 라벨 "환영합니다!"로 바꾸기
5. 모든 위젯(QLineEdit, QPushButton, QLabel)이 메인 창 안에 올바르게 속하도록 부모-자식 관계를 설정하고 적절한 위치(move)를 잡아줍니다.